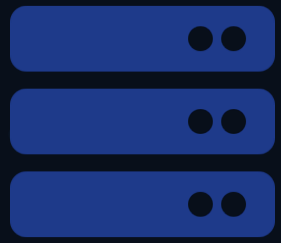
TECHNICAL TRAINING COURSE

Content and Lifecycle in JEE

JAX-RS 2.x · Tomcat 9.0

- Reading Web Resources · Finding JEE Components
- Finding Databases · Connection Pooling
- Security Contexts · Bearer Token Auth

Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 | JNDI/JDBC



COURSE AGENDA

4 Sections | Resources, JNDI, Databases, Security

01 Reading Web Resources

02 Finding Java EE Components

03 Finding Databases

04 Security Contexts

SECTION 1

Reading Web Resources — @Context Injection

@Context ServletContext

Entire web application context. Read files, init-params, attributes.

```
ctx.getResourceAsStream("/WEB-INF/config.xml")
```

@Context HttpServletRequest

Raw request: remote IP, session, request attributes.

```
req.getRemoteAddr(), req.getSession(false)
```

@Context HttpServletResponse

Write response headers directly before JAX-RS does.

```
resp.setHeader("X-Custom","value")
```

@Context UriInfo

Full URI breakdown: path, query params, base URI.

```
ui.getAbsolutePathBuilder().path(id).build()
```

SECTION 1

ServletContext — Reading Files and Init-Params

```
@Path("/config")
public class ConfigResource {

    @Context private ServletContext ctx;

    @GET @Path("version") @Produces(TEXT_PLAIN)
    public String version() {
        return ctx.getInitParameter("app.version");    // from web.xml <context-param>
    }

    @GET @Path("template") @Produces(TEXT_HTML)
    public Response template() throws IOException {
        InputStream is = ctx.getResourceAsStream("/WEB-INF/templates/welcome.html");
        if (is == null) return Response.status(404).build();
        return Response.ok(new String(is.readAllBytes(), UTF_8)).build();
    }
}
```

`getInitParameter(name)`	Reads a <context-param> from web.xml
`getResourceAsStream(path)`	Opens a file relative to the WAR root. Path must start with /.
`getClass().getResourceAsStream(path)`	Opens a file from the Java classpath (src/main/resources)

SECTION 2

Finding JEE Components — JNDI

Java Naming and Directory Interface: the registry for all managed JEE resources

java:comp/env/jdbc/shopDB — JDBC DataSource

```
Context ic = new InitialContext(); ic.lookup("java:comp/env/jdbc/shopDB");
```

java:comp/env/jms/orders — JMS Queue

```
Context ic = new InitialContext(); ic.lookup("java:comp/env/jms/orders");
```

java:comp/env/mail/session — Mail Session

```
Context ic = new InitialContext(); ic.lookup("java:comp/env/mail/session");
```

java:comp/env/app/schema — Custom bound object

```
Context ic = new InitialContext(); ic.lookup("java:comp/env/app/schema");
```

SECTION 2

Configuring JNDI Resources — Tomcat context.xml

```
<!-- src/main/webapp/META-INF/context.xml -->
<Context>

  <!-- JDBC DataSource (connection pool) -->
  <Resource name="jdbc/shopDB"
            auth="Container"
            type="javax.sql.DataSource"
            driverClassName="org.h2.Driver"
            url="jdbc:h2:mem:shopdb;DB_CLOSE_DELAY=-1"
            username="sa" password=""
            maxTotal="10" maxIdle="5" maxWaitMillis="-1"/>

  <!-- Named string constant -->
  <Environment name="app.version"
               type="java.lang.String"
               value="2.4.1" override="false"/>

</Context>

// Accessed as:  java:comp/env/jdbc/shopDB
//               java:comp/env/app.version
```

SECTION 3

Finding Databases — DataSource and Connection Pooling

```
// Correct pattern — cache DataSource, not Connection
@Path("/products")
public class ProductResource {

    private DataSource ds;

    @Context
    public void setServletContext(ServletContext ctx) {
        try { Context ic = new InitialContext();
              ds = (DataSource) ic.lookup(
                  "java:comp/env/jdbc/shopDB"); }
        catch (NamingException e) {
            throw new RuntimeException(e); }
    }

    @GET @Produces(TEXT_PLAIN)
    public String list()
            throws SQLException {
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(
               "SELECT * FROM products");
             ResultSet rs = ps.executeQuery()) {
            // ... build list ...
        }
    }
}
```

Golden Rules

- Cache DataSource — lookup is cheap
- NEVER cache a Connection — not thread-safe
- Always use try-with-resources for Connection, PreparedStatement, ResultSet
- On exception: conn.rollback() then re-throw

SECTION 3

Manual Transactions in Tomcat

Tomcat 9 has no JTA — manage commit/rollback explicitly with JDBC

```
@POST @Consumes(TEXT_PLAIN)
public Response createOrder(Order order) {
    try (Connection conn = ds.getConnection()) {
        conn.setAutoCommit(false);   // start transaction
        try {
            insertOrder(conn, order);
            insertOrderLines(conn, order.getLines());
            reserveStock(conn, order.getLines());
            conn.commit();           // all-or-nothing — success
            return Response.status(201).entity(order).build();
        } catch (Exception e) {
            conn.rollback();         // undo all changes
            throw new InternalServerErrorException(e.getMessage(), e);
        }
    } catch (SQLException e) {
        throw new InternalServerErrorException(e.getMessage(), e);
    }
}

// try-with-resources ensures conn.close() runs even if commit() throws
```

SECTION 4

Security Contexts — Authentication vs Authorisation

Authentication

Who is the caller? Validate credentials → establish identity.

```
401 Unauthorized + WWW-Authenticate: Bearer realm="api"
```

Authorisation

Can this caller do this? Check roles/permissions after identity is known.

```
403 Forbidden (authenticated but insufficient permissions)
```

SECTION 4

Bearer Token Authentication Filter

```
@Provider
@Priority(Priorities.AUTHENTICATION)   // runs before resource methods
public class BearerTokenFilter implements ContainerRequestFilter {

    @Override public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);

        if (auth == null || !auth.startsWith("Bearer ")) {
            ctx.abortWith(Response.status(401)              // 401 Unauthorized
                .header(WWW_AUTHENTICATE, "Bearer realm=api")
                .entity("error=Missing token\nstatus=401\n").build());
            return;
        }

        UserPrincipal principal = validateToken(
            auth.substring("Bearer ".length()).trim());

        if (principal == null) {
            ctx.abortWith(Response.status(401)              // 401 Invalid
                .entity("error=Invalid token\nstatus=401\n").build());
            return;
        }

        ctx.setSecurityContext(                             // install SecurityContext
            new BearerSecurityContext(principal,
                ctx.getSecurityContext().isSecure()));
    }
}
```

SECTION 4

Declarative Authorisation — @RolesAllowed

```
// Requires RolesAllowedDynamicFeature registered

@Path("/admin")
public class AdminResource {

    @GET @Path("users")
    @RolesAllowed("ADMIN")
    public List<User> listUsers() { ... }
    // → 403 if caller is not ADMIN

    @GET @Path("health")
    @PermitAll
    public Response health() {
        return Response.ok(...).build();
    }
    // → no authentication required

    @DELETE @Path("system")
    @DenyAll
    public Response system() { ... }
    // → always 403

    @GET @Path("reports")
    @RolesAllowed({"ADMIN","MANAGER"})
    public List<Report> reports() { ... }
    // → ADMIN or MANAGER allowed
}
```

SecurityContext API

`getUserPrincipal()`

Returns the authenticated Principal (or null)

`isUserInRole(role)`

Delegates to BearerSecurityContext.isUserInRole()

`isSecure()`

True if the connection uses HTTPS

`getAuthenticationScheme()`

e.g. "Bearer" or BASIC_AUTH

SECTION 4

@Secured — Selective Authentication with @NameBinding

```
// 1. Define the binding annotation
@NameBinding
@Retention(RUNTIME)
@Target({TYPE, METHOD})
public @interface Secured {}

// 2. Apply to the filter
@Provider @Secured
@Priority(AUTHENTICATION)
public class BearerTokenFilter
    implements ContainerRequestFilter { ... }

// 3. Apply selectively to resources
@Path("/orders")
public class OrderResource {

    @GET            // public — no auth
    public List<Order> listPublic() { ... }

    @POST
    @Secured        // requires Bearer token
    public Response create(
            Order order,
            @Context SecurityContext sc) {
        String user = sc.getUserPrincipal().getName();
        order.setCreatedBy(user);
        return orderService.save(order);
    }
}
```

```
// Full authentication flow:

// 1. POST /orders
//    Authorization: Bearer <token>

// 2. @Priority(AUTHENTICATION) filter
//    BearerTokenFilter.filter() runs

// 3. Token valid?
//    → ctx.setSecurityContext(
//        new BearerSecurityContext(
//          principal, isSecure))

// 4. Token absent or invalid?
//    → ctx.abortWith(401)

// 5. @RolesAllowed check:
//    sc.isUserInRole("user")?
//    → false → 403

// 6. Execute resource method
//    sc.getUserPrincipal().getName()
//    → "alice"
```

COURSE SUMMARY

Key Takeaways

01 @Context injects ServletContext, HttpServletRequest, SecurityContext, UriInfo, and other container objects.

02 getResourceAsStream('/WEB-INF/...') reads WAR files; getClass().getResourceAsStream reads classpath files.

03 JNDI (java:comp/env/...) is the standard registry for DataSources, JMS, and custom resources in Tomcat.

04 Cache the DataSource — never cache a Connection. Use try-with-resources for Connection, PreparedStatement, ResultSet.

05 In Tomcat, manage transactions manually: setAutoCommit(false), commit(), rollback().

06 Authentication filter → setSecurityContext(). @RolesAllowed + @PermitAll + @DenyAll for declarative auth.

07 Use @NameBinding + @Secured to apply authentication selectively to specific endpoints rather than globally.

Thank You

Questions & Lab Time

Lab exercises:

- Lab 1: Reading web resources via @Context ServletContext
- Lab 2: JNDI lookups — binding and finding components
- Lab 3: DataSource, connection pooling, manual transactions
- Lab 4: Bearer token filter + SecurityContext + @RolesAllowed